STAT 3080: From Data to Knowledge

Jeffrey Woo

2026-07-23

Contents

Preface

Course Overview and Goals

This course focuses on a simulation-based inference approach to concepts taught in elementary statistics. It focuses on using simulations to investigate the operating characteristics of common inferential procedures in statistics, so you can make practical decisions on when to use these procedures to learn about the data at hand.

For example, you may have heard that a hypothesis test for the mean from one sample assumes the data come from a normal distribution, or that the sample size is "large enough". You may have wondered what constitutes being "large enough", and you may have learned or read that a sample size of 25 or 30 will be considered "large enough". The true answer is actually "it depends", as various other factors play a role.

You will learn to design and run simulations to develop a more nuanced understanding of how reliable these inferential procedures are when the assumptions they are based on are not met. It turns out that there are mathematical proofs that can answer such questions, but they are usually fairly complicated, so we will use simulations in place of these mathematical proofs.

In order to design such simulations, a fair amount of knowledge with coding using R will be needed, which you will learn in the first portion of this course. This course is not a programming course; it is simulation-based inference using R.

Upon successful completion of this course, you will be able to:

1. Use simulation to study operating characteristics of statistical methodologies.
2. Understand and apply appropriate statistical methodologies under various data scenarios.
3. Use simulation to estimate statistical quantities which are difficult to compute.
4. Use simulation to assess the quality of statistical estimators.

5. Communicate results from simulations in writing, based on principles of reproducible research.

Background Needed for the Course

The official prerequisites are:

1. Intro statistics (One of STAT 1100, STAT 1120, STAT 2020, STAT 2120, STAT 3120, APMA 3110, APMA 3120).
2. Intro programming (One of STAT 1601, STAT 1602, STAT 3250, CS 1110, CS 1111, CS 1112, CS 1113).

The course is presented assuming you are familiar with inferential procedures associated with one- or two-sample means and proportions, as well as basic programming skills. The course will not cover these.

Required Software

The software introduced and featured in this course is R. RStudio is a software that runs R with additional user-friendly features. RStudio will be used for all in-class R demonstrations. Both R and RStudio are free for download.

- R will need to be downloaded first.
- Then downloaded RStudio.

You are **highly encouraged** to use R Markdown to create PDF documents for homeworks and the project, which uses a type-setting platform called LaTeX. LaTeX can be installed by running the following two commands in the R Console:

```
install.packages("tinytex")
tinytex::install_tinytex()
```

Chapters

The chapters for the book is as follows:

- Chapters ??? focus on core R skills needed: .
- Chapters ??? focus on

Reporting Issues with Course Notes

If you find any issues (typos, formatting, etc), please report them at my GitHub page. Please be as specific as you can, by specifying the section and paragraph where the issue is found.

Serious issues that need my immediate attention should be posted on our Canvas page on Piazza or emailed to me.

Chapter 1

R Basics

1.1 Overview of R

R is an open-source programming language and software environment specifically designed for statistical computing, data analysis, and graphical visualization. It is widely used by data scientists, statisticians, and researchers to clean, manipulate, and model data. R is maintained by volunteers. All users can access and modify source code.

While R has a basic default interface, users typically write code using integrated development environments (IDEs) such as RStudio, Positron, VS Code, which can provide additional user-friendly features. RStudio will be used in this course.

1.1.1 RStudio Interface

Open RStudio. You will see three panes:

- **Console** (left): where commands are executed
- **Environment / History** (top-right:) objects you have created and the commands you have run.
- **Files / Plots / Packages / Help** (bottom-right): files in working directory, plots created, packages installed, help page.

We will focus on the console for the time being.

1.2 Commands

In order to get R to do what you want it to do, you will need to write a **command**. R is ready for a new command when you see a `>` in the **console**. If your command is incomplete, R will show a `+` instead of a `>`. You can either

complete the command after the `+` or use the ESC key to abort. You will also see the output of your command (if any) in the console. For example:

```
1+2
```

```
## [1] 3
```

Notice a number in brackets before the answer? The number indicates index position of the first element displayed on that specific line.

R also ignores white space between components of a command, so typing `1+2` is the same as typing `1 + 2`.

```
1 + 2
```

```
## [1] 3
```

You can also add **comments** to R code by using the `#` sign. Everything to the right of the `#` is ignored by R. Comments are useful to explain what has been coded and why to yourself or to collaborators.

```
1+2 ## what is the sum of 1 and 2?
```

```
## [1] 3
```

I **highly discourage** you to enter your code directly into the R console. Code typed into the console is difficult to be saved or edited. Any meaningful work that you will use R for will likely need to be saved and edited.

1.3 Working Directory

Before talking about a better way to enter R commands, you will need to set up a **working directory**. The working directory is the path to the folder on your system (typically, the computer you are using) to store files generated during an R session and any files that you save. This is also the folder where R will search for files.

1.3.1 Finding the working directory

Typing `getwd()` will display the path of the working directory:

```
getwd()
```

```
## [1] "C:/Users/jeffr/Dropbox/UVA/book_stat3080/stat3080"
```

1.3.2 Changing the working directory

To change the working directory, use `setwd()`, with the path place as an argument:

```
setwd("C:/Users/jeffr/Dropbox/UVA/book_stat3080/stat3080/HWs")
```

I do not recommend using `setwd()`:

- Typing the path is fairly cumbersome.
- Your code will not run if you run the code on another computer (e.g. when sharing with a collaborator).

Therefore, I prefer to use RStudio Projects.

1.3.3 RStudio Projects

An **RStudio Project** is a folder with a `.Rproj` file that in it. There are a couple of ways to create an RStudio Project.

1. If you have never created one before:

- Open RStudio.
- Click on File, then New Project.
- A few options will be presented. The options that you will most likely use are:
  - New Directory: use this to start in a fresh folder.
  - Existing Directory: use this to work in an existing folder.
- If you are a creating a new directory, select New Project. Then you will enter the name of your project and choose a path for the folder.

If you open the folder associated with the created working directory, you will find a file with a `.Rproj` extension created (its name should be the name of the project that you gave in the last step above).

Now, close your RStudio. Navigate to the project folder on your computer and double-click the `.Rproj` file. Your working directory will be where this `.Rproj` file is located. You can confirm using `getwd()`, or by looking at the pane called "Files / Plots / Packages / Help". Click on the files tab and it will display the path of your working directory, and you can also see the other files in this folder.
2. If you have created an RStudio Project before and have a file with a `.RProj` extension, you can just copy and paste this file and place it into another folder to make it your working directory. This is the approach I almost exclusively use.

I highly recommend that you create a folder on your computer that is exclusively for this class. You should also create separate subfolders for each assignment.

Key idea: The working directory is the folder on your computer where all files that R generates will be saved and where R will search for files.

Practice: create an RStudio Project on your own.

1.4 R Scripts

You should type R commands in an R **script** instead of the R console. Commands can be saved and edited in an R script.

To create a new R script, click on File, New File, R Script. You will now have a new pane on the top-left with a tab labeled with a name similar to Untitled. You have created an untitled R script.

An R script looks like a notepad. Type your commands into this script. Each command should be a separate line. To execute commands, highlight the commands, and then click on Run or press Ctrl + Enter.

To save your R script, click on the Save icon and name your script. You have now saved an R Script, and it should appear in your working directory.

To open a saved R script, open the `.Rproj` file in the working directory, and you can click on the script under the tab called Files.

1.5 Basic Math Operations

R can be used as a calculator by using the standard arithmetic operators: + - `+`: addition - `-`: subtraction - `*`: multiplication - `/`: division - `^`: power

R is like any calculator in that these operators follow the standard order of operations. Some examples:

```
15+10
12/6
0.7*5
3^3
15+10*3
(15+10)*3
```

1.6 Assignments and Variables

In order to write useful and effective R code, You need to be able to store items to be used later. Anything that is stored is called an object. An object can be a single value resulting from a calculation or a collection of information such as a table of linear regression coefficients.

A **variable** is a memory location for the object, which can be thought of as a label for the object. Variables are created when the object is **assigned** to it, so there is no need to declare the variable first.

Note: the term variable as defined above comes from computer science. In statistics, the term variable means something different (any characteristic that can be measured).

Variable names can consist of letters, numbers, periods, and underscores. There are a few limitations to how these types of characters can be combined. The following are characteristics of all variable names:

1. Variable names cannot begin with a number.
2. Variable names cannot contain spaces.
3. Variable names can begin with a period as long as they are not followed by a number.
4. Since R is case-sensitive, `x` and `X` are different variable names. Wherever possible, it is best practice to avoid using the names of existing functions (ie. mean, sd, log) and predefined variables (ie. pi) to avoid confusion for anyone reading your code and possibly R itself.

It is also best practice to use variable names that are short and descriptive of the information contained in the object.

Once you have decided on a variable name for the object, you will need to assign the object to the variable by using either the assignment operator `<-` or `=` (the first is the more commonly used in R). I would discourage using `=` as an assignment operator as it is also used when specifying values for arguments in functions (see section 1.7.1.)

To store the sum of 10 and 5 and the rounded value of 5.7 (to whole number) into variables called `total1` and `value1` respectively:

```
total1 <- 10+15
value1 <- round(5.7,0)
```

Notice that these variables and their values are displayed in the environment tab in the pane at the top-right.

R will recall or show the object stored in a variable when the variable name is submitted as a command.

```
total1
```

```
## [1] 25
```

```
value1
```

```
## [1] 6
```

Stored objects can be used in subsequent operations. For example:

```
total1/5
```

```
## [1] 5
```

```
total1/value1
```

```
## [1] 4.166667
```

Any object that you would like stored needs to be given a variable name.

```
ratio1 <- total1/value1
```

Variables can be overwritten by storing a new object using the same variable name.

```
ratio1 <- total1/2
```

1.7 Functions

R has an extensive list of existing functions that can be used. Each function has a specific name that is followed by a set of parentheses. Using a function calls a specific collection of code to be executed. Generally the code that will be executed needs specific arguments or inputs to run correctly and the user provides these values in the parentheses that follow the function name.

Functions can be used for basic calculations, for complex procedures, and many other things in between. You can also write your own functions.

R is case-sensitive, so functions that you want to use must be typed exactly as it was defined.

Some common functions are listed below:

```
log(2.7) ## log with base e, sometimes written as ln
exp(1) ## e to power of 1
sqrt(9) ## squareroot of 9
factorial(3) ## 3!
abs(-5) ## absolute value
sum(2,4,6) ## summing up all these numbers
floor(5.7) ## round down to nearest integer
ceiling(5.7) ## round down to nearest integer
round(5.7) ## round to nearest integer
round(5.727,2) ##round to 2 decimal places
```

1.7.1 Documentation

Typing `?` and then the name of a function will open up the help page (called R documentation) associated with that function. The help page will appear under the Help tab in the bottom-right pane. Type `?log` to read the help page for the `log()` function.

The sections of the help page that I find useful:

- Description: describes what the function does.
- Usage: tells us what arguments need to be provided, and if arguments have defaults.
- Arguments: describes the arguments.
- Examples: some examples.

Practice: after reading the documentation for `log()`, use R to find the value of $\log_3(5)$.

Note: In the top-left corner of the help page, the name of the function is shown, along with the package it is from in braces. So the `log()` function comes a package called `base`.

1.7.2 Working with common probability distributions

Some common probability distributions (e.g. normal, binomial, etc.) are defined in R. Each distribution has a general function name, for example:

- norm: normal distribution
- binom: binomial distribution
- t: t distribution
- F: F distribution
- chisq: χ^2 distribution

Each distribution has 4 functions associated with it in R. The letter that precedes the distribution name tells R what specifically to do with that distribution. These letters are:

- `d`: gives the value of the probability density (or mass) function.
- `p`: gives the cumulative probability of the distribution at the point specified.
- `q`: determines the value within the distribution associated with the cumulative probability specified.
- `r`: generates random numbers from the distribution.

1.7.2.1 Example 1: Normal distribution

Suppose $X \sim N(1, 9)$, i.e. a normal distribution with mean 1 and variance 9.

- To find $P(X \leq 2)$, we use:

```
pnorm(2, mean=1, sd=3)
```

```
## [1] 0.6305587
```

- To find the value of X associated with the 25th percentile:

```
qnorm(0.25, mean=1, sd=3)
```

```
## [1] -1.023469
```

- To simulate 10 random draws from X, we use:

```
rnorm(10, mean=1, sd=3)
```

```
##  [1] -1.0089300 -3.5732585  0.8976058  3.6723282  0.0700096  2.9762124
##  [7]  0.7046069 -0.5976909  2.8329462  0.3401275
```

Note: you are likely to get a vector of 10 numbers that are different than the ones shown.

1.7.2.2 Example 2: Binomial distribution

Suppose $Y \sim Bin(10, 0.6)$, i.e. a binomial distribution with $n = 10$ and $p = 0.6$.

- To find $P(Y = 3)$, we use:

```
dbinom(3, size=10, prob=0.6)
```

```
## [1] 0.04246733
```

To find $P(Y \leq 3)$, we use:

```
pbinom(3, size=10, prob=0.6)
```

```
## [1] 0.05476188
```

Add Practice questions:

1.8 Packages

The functionality of R is broken up into various packages. There are four packages included in an R download: base, graphics, stats, utils. These four packages are sometimes collectively called "base R packages". An R **package** is a collection of R functions, corresponding documentation, and datasets.

However, statistical methodologies advance, and additional functions beyond those that exist in base R packages are needed. Independent statisticians create packages to solve more modern problems and have contributed them for everyone to use. These packages are sometimes called "contributed packages".

In order to use what is in a contributed package, that package needs to be installed first, and then loaded. You do not need to install or load base R packages.

In order to install packages, you must have an internet connection. Packages only need to be installed once on a computer unless the version of R is updated. We use the function `install.packages()`. As an example, to install the package called `palmerpenguins`:

```
install.packages("palmerpenguins")
```

You can then load the package in order to able to use the functions from that package, by using the `library()` function:

```
library(palmerpenguins)
```

Key idea: packages only need to be installed once (unless you update R). Packages must be loaded prior to an R session using `library()`.

1.9 Types, Data Structures, Classes

In R, **type** refers to how an object is stored in memory. You can refer to this table for a list of types in R. To find the type of an R object, use the `typeof()` function:

```
typeof(total1)
```

```
## [1] "double"
```

Data structures are used to store and organize values. They are classified based on dimensionality and homogeneity.

- A single value is considered to have zero dimensions and is called a scalar.
- A data structure that is either a single column or a single row is considered to have one dimension.
  - If everything recorded in a one-dimensional data structure is of the same type, then it is called a **vector**.
  - If the items recorded in a one-dimensional data structure have varying types, then it is called a **list**.
- A data structure that contains multiple rows and multiple columns is considered to have two dimensions.
  - If the contents of a two-dimensional data structure are homogeneous, then it is called a **matrix**.
  - If the contents of a two-dimensional data structure are heterogeneous, then it is called a **data frame**.
- An **array** is a data structure that is homogeneous with one or more dimensions.

This information is summarized in Table 1.1 below:

Table 1.1: Classification of R Data Structures

Dimension	Homogeneous (Same Type)	Heterogeneous (Diff Type)
One	Vector	List
Two	Matrix	Data Frame
>2	Array	-

Data structures are created either when data are read in from external files or when data are entered manually. Next, we will see how to create them manually.

1.9.1 Vectors

The concatenate function, `c()`, is commonly used to create a vector. The inputs for this function are the data that you would like stored in the vector in the order in which you would like them stored. For example:

```
x <- c(5,10,15,20,25)
x
```

```
## [1]  5 10 15 20 25
```

```
y <- c(3,6,9,12,15)
y
```

```
## [1]  3  6  9 12 15
```

The concatenate function can also be used to add a value to a vector or to combine preexisting vectors.

```
x2 <- c(x,30)
z <- c(x,y)
z
```

```
##  [1]  5 10 15 20 25  3  6  9 12 15
```

It is important to remember that the order in which the objects are specified is the order in which they will be saved in the vector.

```
c(y,x)
```

```
##  [1]  3  6  9 12 15  5 10 15 20 25
```

1.9.1.1 Pattern vectors

There are times when you may need to create a vector that contains a sequence of numbers. There are a few ways to create this type of vector.

1. If you are wanting to create a vector that contains a sequence of consecutive integers, then you can use the syntax a:b to create a vector that starts at a and increases or decreases by one until b.

```
1:5
```

```
## [1] 1 2 3 4 5
```

```
5:1
```

```
## [1] 5 4 3 2 1
```

```
-1:-5
```

```
## [1] -1 -2 -3 -4 -5
```

2. If you are wanting a sequence where the values have a constant difference other than one, then you can use the sequence function, `seq()`. This function takes three arguments: from, to, by. These inputs specify the first and last number of the sequence as well as the difference between each value in the sequence.

```
seq(from=1, to=5, by=0.5)
```

```
## [1] 1.0 1.5 2.0 2.5 3.0 3.5 4.0 4.5 5.0
```

There is another optional input for the sequence function: length. The length option specifies the length that you want your vector to be or how many values you want contained in the vector. The sequence function will use this value to determine a constant difference between each of the values that yields the number of values that you specify.

```
seq(1,5, length=9)
```

```
## [1] 1.0 1.5 2.0 2.5 3.0 3.5 4.0 4.5 5.0
```

3. Another function that creates a useful type of vector is the replicate function, `rep()`. This function takes two arguments: vector and number of replicates. These specify what you would like R to repeat and how many times. Depending on the format of what you provide in the times option, this function will either repeat the given vector a specified number of times or repeat the elements of the vector a specified number of times.

```
rep(1:5, 2)
```

```
##  [1] 1 2 3 4 5 1 2 3 4 5
```

```
rep(1:5, c(1,2,3,2,1))
```

```
## [1] 1 2 2 3 3 3 4 4 5
```

```
rep(1:5, c(2,2,2,2,2))
```

```
##  [1] 1 1 2 2 3 3 4 4 5 5
```

```
rep(1:5, each=2)
```

```
##  [1] 1 1 2 2 3 3 4 4 5 5
```

1.9.2 Matrices

The `matrix()` function can be used to create a matrix. This function takes four arguments: vector, nrow, ncol, and byrow. These arguments instruct R what values to put in the matrix, how many rows and columns the matrix should have, and whether values should be assigned across the rows or down the columns. The default for the byrow option is false, meaning that the values will be assigned down the columns if the byrow option is not provided.

```
matrix(c(5,10,15,20,25,3,6,9,12,15), nrow=2, ncol=5, byrow=TRUE)
```

```
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    5   10   15   20   25
## [2,]    3    6    9   12   15
```

Another way to create a matrix is to use either the `cbind()` or `rbind()` function. Both of these functions use vectors as inputs. The `cbind()` function will bind these vectors together as columns and the `rbind()` function will bind these vectors together as rows.

```
rbind(x,y)
```

```
##   [,1] [,2] [,3] [,4] [,5]
## x    5   10   15   20   25
## y    3    6    9   12   15
```

```
cbind(x,y)
```

```
##       x  y
## [1,]  5  3
## [2,] 10  6
## [3,] 15  9
## [4,] 20 12
## [5,] 25 15
```

As with the concatenate function, R will bind the vectors together in the order you specify.

1.9.3 Classes

Before going into the remaining data structures, you need to learn what classes are. In R, **class** refers to how an object behaves with generic functions. There are 3 main classes in R: character, numeric, and logical.

- Character: Represents text strings, symbols, or words. Must always be enclosed in quotes.
- Numeric: Represents real numbers.
- Logical: Represents Boolean values for binary conditions.

The vectors and matrices created in sections 1.9.1 and 1.9.2 are numeric.

```
class(y)
```

```
## [1] "numeric"
```

To create a logical vector:

```
log_vect <- c(TRUE, FALSE, T, F)
## Note that T and F are equivalent to TRUE and FALSE resp
```

To create a character vector:

```
char_vect <- c("ByOne", "ByTwo", "ByThree", "ByFour", "ByFive")
```

The `class()` function can be used to find out the class of an object, for example:

```
class(log_vect)
```

```
## [1] "logical"
```

Alternatively, functions such as `is.character()`,is.numeric(),is.logical()‘ can be used to test for the class:

```
is.numeric(log_vect)
```

```
## [1] FALSE
```

If a vector or matrix contains multiple types of data, an error message is unlikely to show up. R will modify the data to all be of the same type according to the following hierarchical structure:

1. character
2. numeric
3. logical

```
c(TRUE, 5)
```

```
## [1] 1 5
```

```
c(TRUE, "ByFive", 5)
```

```
## [1] "TRUE"   "ByFive" "5"
```

```
mult_mat <- cbind(char_vect,x,y)
mult_mat
```

```
##      char_vect x    y
## [1,] "ByOne"   "5"  "3"
## [2,] "ByTwo"   "10" "6"
## [3,] "ByThree" "15" "9"
## [4,] "ByFour"  "20" "12"
## [5,] "ByFive"  "25" "15"
```

1.9.3.1 Example 1: Mixing numeric with character

The most common kind of mixing of classes usually involve a data structure containing a mix of numeric and character, for example:

```
mixed <- c(1, 2, "3")
mixed ## notice all characters, since they are in quotes
```

```
## [1] "1" "2" "3"
```

```
class(mixed)
```

```
## [1] "character"
```

Trying to add the numbers in the variable `mixed` results in an error:

```
sum(mixed)
```

In this example, we can **coerce** all entries to become numeric:

```
mixed <- as.numeric(mixed) ## overwrite existing variable
sum(mixed)
```

```
## [1] 6
```

A situation like this example sometimes happens when working with an external data file and the person who entered the data made a mistake. This is a common reason why a mathematical operation fails as one character changes the entire vector to be character.

Note: Coercing an object to change will not always yield a result that makes sense.

1.9.3.2 Example 2: Math operations with logical

Interestingly, math operations can be performed on logicals. `TRUE` and `FALSE` is considered to be 1 and 0 respectively.

```
sum(log_vect)
```

```
## [1] 2
```

```
mean(log_vect)
```

```
## [1] 0.5
```

This will be extremely useful later on when working with simulations, as we often want to evaluate the proportion of times certain conditions are met.

1.9.4 Data Frames

Data frames are commonly used to store data. There is typically a row for each observation and a column for each variable measured on the observations (think of an EXCEL spreadsheet). A data frame can be created using the `data.frame()` function. The arguments to this function are the vectors that you would like to be in the data frame.

```
mult_data <- data.frame(char_vect,x,y)
mult_data
```

```
##   char_vect  x  y
## 1     ByOne  5  3
## 2     ByTwo 10  6
## 3   ByThree 15  9
## 4    ByFour 20 12
```

```
## 5    ByFive 25 15
```

Notice you can also view the data frame by clicking on its name in the Environment tab in the top-right.

1.9.5 Lists

Lists are the most flexible type of object in R. Lists are similar to vectors in that there is one row containing multiple values. However, in a list, those values can be objects of varying types and sizes. A list is created using the list function, `list()`. The arguments specify what you would like in each element of the list.

```
mult_list <- list(mult_data, z, log_vect, 7)
mult_list
```

```
## [[1]]
##   char_vect  x  y
## 1     ByOne  5  3
## 2     ByTwo 10  6
## 3   ByThree 15  9
## 4    ByFour 20 12
## 5    ByFive 25 15
##
## [[2]]
##  [1]  5 10 15 20 25  3  6  9 12 15
##
## [[3]]
## [1]  TRUE FALSE  TRUE FALSE
##
## [[4]]
## [1] 7
```

There are testing and cohersion functions for all of the types of data structures. As with the classes of data, cohersion may not yield a valid result. Some examples are shown below:

```
is.vector(x)
as.vector(mult_list)
as.data.frame(mult_mat)
```

1.10 Other Practical Advice with R

1.10.1 The Workspace

When you are ready to exit R, you may be asked whether you want to save the workspace image into a `.RData` file. The workspace is not the same as the R script. It refers to the objects in the current environment. My recommendation is not to save the workspace.

This keeps your environment fresh the next time you open R. You will then re-run your R script to reproduce your results. Such practice is consistent with principles associated with **reproducible research**. An environment that contains objects from previous workspaces will interfere with the current workspace and must be avoided.

A situation where you may want to save the workspace is if your code takes a long time to run, and you want to avoid having to run your code again. This is unlikely to be a situation in this class.

To remove this prompt, go to Tools, then Global Options. Under the General tab on the left, and the Basic tab on top, uncheck all boxes listed under R Sessions, Workspace, and History. For the option Save workspace to .RData on exit, choose never.

If you have saved the workspace before, find the .RData file in your working directory, and delete it.

1.10.2 The Environment

Always check that your environment is empty (clear the environment) whenever you start working with R. This will ensure that previously declared variables will not influence the results from your code.

You should also clear the environment if you edited your code after executing it. You do not want previous mistakes or version of the code to influence your results.

To clear the environment to make it empty:

1. Execute the code `rm(list = ls())`, or
2. Click on the broom icon on the top-right under the Environment tab.

Chapter 2

R Markdown

2.1 Why Use R Markdown

Imagine you are in this scenario: you have a homework assignment where you are supposed to produce data visualizations and answer some questions based on a data set using R. Your teacher needs to assess if you know what code to write and that you are providing your commentary on the R output. Therefore, you need to produce a document that show the following:

1. R code.
2. Relevant graphs.
3. Your interpretations of the graphs.

Not only these, but these have to be presented in a cohesive and organized manner.

Providing just an R script will not help, as it only shows your R code and you could provide your interpretations as comments. However, your graphs are not shown. Your teacher does want to produce the graphs by running your R code as it will take a lot of time to run code for all students in the class.

You might think of copying and pasting your R code, your graphs from the output, and type you interpretations into a word document. This is doable but it will take you more time than you think, and the document is likely to look sloppy.

Is there a tool that will help you weave your code, R output, and your interpretations into a neat document? This is what R Markdown does!

Note: R Markdown is not the only tool that allows you to do this. Quarto is another tool. R Markdown and Quarto can be thought of as variants of Markdown. Markdown is a language for formatting plain text. R Markdown and Quarto expand on Markdown.

2.1.1 Reproducible Research

Another reason for using R Markdown is to adhere to **reproducible research** principles. According to Claerbout and Karrenbach (1992), the definition of reproducible research is "Authors provide all the necessary data and the computer codes to run the analysis again, re-creating the results."

Since your document provides the R commands, readers can simply execute the commands to verify your output. Reproducible research improves transparency and builds trust in your work.

Note: The Turing Way provides an overview of the terms reproducible, replicable, robust, and generalizable for research. We will not go into the details for this class but it is worth your time to think about these.

Chapter 3

Blank Canvas

See Sections **??**, 1.7.1.

See equation (**??**).

See Figure 3.1, and 3.2 and Table **??**.

```
plot(cars)
```

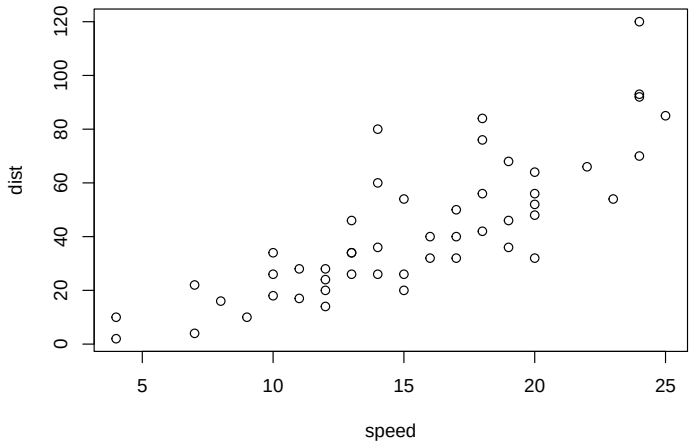

Figure 3.1: Car

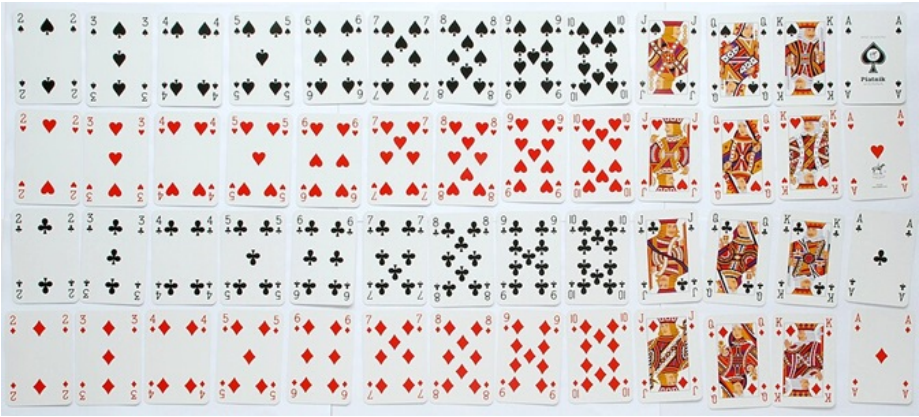

Figure 3.2: Sample Space of Drawing One Card from Standard Deck. Picture from wikipedia

Bibliography

Claerbout, F. and Karrenbach, M. (1992). Electronic documents give reproducible research a new meaning. In *SEG Technical Program Expanded Abstracts 1992*.